

# Beacon Friction Scanner — Ashish Jain Kothari

## Beacon Friction Scanner (MVP)

Store pick flagged, not yet explicitly approved by Ashish. The spike (docs/spike-store-selection.md) picked **bluetokaicoffee.com** (fallback: **gymshark.com**) against the criteria in docs/PRD.md §6, and — per the work order’s own contingency — proceeded without waiting on a reply. Full reasoning, including a criterion reinterpretation, is in docs/decisions-during-build.md (D-01). The real, live scan below (task I.3) has since run successfully against this store, at Ashish’s own request. Swap the host if a different store is preferred; re-running the spike against another candidate is cheap.

Beacon scans a Shopify store’s public storefront data and tells a merchant which sold-out products are leaking the most demand, in what order to restock them, and where else in the shopper journey that intent is going missing.

## Headline finding

From two real, live scans against **bluetokaicoffee.com** — 2026-09-22 and again 2026-09-23, ~14h apart — with `beacon diff` run between them (full report: [out/EXAMPLE/report.html](#)):

- **199 of 959 variants** are sold out across the 190 scanned products, and **20 products** are leaking that demand while still sitting in a merchandised collection (bestsellers, coffee-beans, etc.) — the clearest form of intent hitting a dead end.
- Intent capture: **0 of 19** sampled sold-out product pages had no back-in-stock signup — this store already runs a generic capture mechanism (no swym marker found, though), so the gap for Swym here is adoption, not absence.
- Top of the restock list: **Amruthavarshini Estate** (idx 0.760, 39/39 pack-size variants sold out, position 5 in “Best Coffee Beans in India”) — see the top-15 table in the report for the rest, each with the components behind its score.
- Diff between the two runs: **zero** variants sold through or restocked (out/EXAMPLE/delta.json) — a genuinely quiet 14-hour window for this store’s inventory, not a stubbed feature; `beacon diff`’s sell-through/restock detection and velocity scoring component are implemented and tested (tests/engine.test.ts), just with nothing to show on this particular pair of runs.

- Spot-check: <https://bluetokaicoffee.com/products/amruthavarshini-estate> was, as of the first scan, genuinely fully sold out and does sit in a merchandised collection — matching the report.

These are indices, not revenue figures — public storefront data has no order-volume signal, so nothing here is stated in currency (see MVP boundary and Assumption A3 below).

199

/959

Variants sold out (of scanned total)

20

Products leaking demand on  
merchandised collections

0

Sold-out PDPs without back-in-stock  
capture

Restock Priority — top 15 of 53

"Restock Priority" is a unitless intent-at-risk index (0–1) built from public storefront signals — never a revenue or currency figure. Click a row for its component breakdown.

|     |                                        |                         |           |
|-----|----------------------------------------|-------------------------|-----------|
| #1  | Amruthavarshini Estate                 | 39/39 variants sold out | idx 0.760 |
| #2  | Kerehaklu Estate                       | 3/3 variants sold out   | idx 0.753 |
| #3  | Assorted 6-Pack                        | 1/1 variants sold out   | idx 0.732 |
| #4  | Wine Barrel Aged Coffee                | 1/1 variants sold out   | idx 0.731 |
| #5  | Raxidi Lobo Estate - Anaerobic Natural | 3/3 variants sold out   | idx 0.705 |
| #6  | Howdia Estate                          | 1/1 variants sold out   | idx 0.686 |
| #7  | Warsanlyngdoh Coffee Collective        | 39/39 variants sold out | idx 0.681 |
| #8  | Riverdale Estate - White Rose          | 1/1 variants sold out   | idx 0.672 |
| #9  | Kerehaklu Estate - Peaberries          | 1/1 variants sold out   | idx 0.668 |
| #10 | Origin Unhurried: Yercaud Pack         | 1/1 variants sold out   | idx 0.668 |
| #11 | MSP Micro Lot Blend                    | 1/1 variants sold out   | idx 0.663 |
| #12 | Mooleh Manay Estate – CM Natural       | 1/1 variants sold out   | idx 0.660 |
| #13 | SKIA Coffee (Pack of 2)                | 1/1 variants sold out   | idx 0.641 |
| #14 | Araku - Viishalakshi FPC               | 3/3 variants sold out   | idx 0.637 |
| #15 | Baarbara Estate - Whiskey Barrel Aged  | 1/1 variants sold out   | idx 0.621 |

Journey friction scorecard

Five shopper-journey stages, each checked against public evidence. A "None" severity chip means the check ran and found no issue, not that the stage was skipped.

Discovery

NONE

Products with no collection placement

0 / 190

No action needed — products with no collection placement found no issues on this run.

No example URLs recorded for this check.

Evidence: products.json cross-referenced against collections<handle>products.json for every recorded collection, excluding "all"/"frontpage"

HIGH

Sold-out products in top collection positions

3

Reorder the collection grid so a fully sold-out product isn't the first thing a shopper sees, or restock it before the next campaign send.

- Amruthavarshini Estate — position 5 in Best Coffee Beans in India, fully sold out
- Wine Barrel Aged Coffee — position 4 in Best Coffee Beans in India, fully sold out
- Assorted 6-Pack — position 8 in Best Sellers NAV, fully sold out

Evidence: collection-<handle>.json position index for fully sold-out products, top 8 positions of any merchandised collection

PDP

MEDIUM

Product pages with fewer than 2 images

32 / 190

Add a second product image — single-image PDPs read as unfinished and convert worse than pages with multiple angles.

- Drop Trial Pack — 1 image
- Terroir Culture by Ratnagiri Estate — 1 image
- Riverdale Estate - Mosto — 1 image

Evidence: products.json "images[]" length per product

HIGH

Product pages with a short description

65 / 190

Expand the description past a couple of sentences — a description under 200 characters gives a shopper nothing to justify the price or answer basic questions.

- Drop Trial Pack — 0 characters
- Koli Berri Estate Subscription — 0 characters
- Kalledevarapura Estate - Pulp Sun Dried Subscription — 0 characters

Evidence: products.json "body\_html" stripped to text, length in characters

HIGH

Colour variants missing their own image

2 / 3

Add a dedicated image for every colour variant — a shopper choosing colour from a swatch with no matching photo is guessing, not deciding.

- COFFEE T-SHIRT Yellow (Pour over) — 5 of 5 variants missing their own image
- COFFEE T-SHIRT Blue (Moka pot) — 5 of 5 variants missing their own image

Evidence: products.json option named Colour/Color; variant "featured\_image" presence ("hasOwnImage")

Variant selection

MEDIUM

Core pack sizes sold out

33 / 144

Prioritise restocking the middle sizes first — the smallest and largest sizes selling through last is normal; the middle going out isn't.

- Baarbara Estate - Whiskey Barrel Aged — core sizes 250g sold out
- Excelsa by Moolleh Manay Estate — core sizes 100g sold out
- Riverdale Estate - Mosto — core sizes 200g sold out

Evidence: products.json size option values, ordered; middle 50% checked for sold-out

Intent capture

NONE

Sold-out PDPs with no back-in-stock capture

0 / 19

No action needed — sold-out PDPs with no back-in-stock capture found no issues on this run.

No example URLs recorded for this check.

Evidence: PDP HTML sampled for highest-leakage sold-out products; searched for swym/back in stock/notify me/restock/klaviyo-bis/bis markers

Trust / checkout

## Quick start

```
npm i
npx beacon scan https://bluetokaicoffee.com
```

This writes a timestamped run to `out/bluetokaicoffee.com/<ISO-timestamp>/: snapshot.json, findings.json, restock_priority.csv, report.html` (open it directly, no server needed), and a terminal summary of the three headline numbers plus the top 10 restock rows. First run takes under five minutes on a normal connection; a `.cache/` directory keeps re-runs fast and under the same request budget.

**Programmatic use** — the CLI is a thin wrapper around the same library call:

```
import { scanStore } from "beacon-scan";

const { findings } = await scanStore("https://
  bluetokaicoffee.com");
console.log(findings.headline);
console.log(findings.restock[0]); // top restock priority row
```

## Where the rest of the exercise lives

| Part                                                           | Location                                                             |
|----------------------------------------------------------------|----------------------------------------------------------------------|
| Example report (offline fixture run — see note above)          | <a href="#">out/EXAMPLE/report.html</a>                              |
| Part 2 — architecture (NL query at 50K-store scale)            | <a href="#">docs/part2-architecture.md</a>                           |
| Part 2 — defensibility (standalone product or Shopify feature) | <a href="#">docs/part2-defensibility.md</a>                          |
| Part 3 — discovery and prioritisation framework                | <a href="#">docs/part3-discovery.md</a>                              |
| AI workflow (how this was built, with Claude Code)             | <a href="#">docs/ai-workflow/README.md</a>                           |
| Loom walkthrough (≤3 min)                                      | <i>not yet recorded</i> — see <a href="#">docs/loom-shot-list.md</a> |
| PDF mirror of this write-up                                    | <i>not yet generated</i> — see <a href="#">scripts/export-pdf.md</a> |

## MVP boundary

This is a deliberately bounded MVP. The following is what it does **not** do, quoted verbatim from the locked PRD (docs/PRD.md §1) — this is the honest list of reasons Swym couldn't use this as-is:

- **N1.** No hosted service, scheduler, multi-store, auth, database server, or UI beyond a static HTML report.
- **N2.** No runtime LLM. The NL-query layer is designed in docs/part2-architecture.md, not implemented.
- **N3.** No Swym data integration; no assumptions about Swym internals.
- **N4.** Scoring is a transparent v0 heuristic with weights in one config file; no calibration, no ML.
- **N5.** Only two friction families are scored in depth (Demand Leakage, Variant/PDP hygiene); the other journey stages get 1–2 cheap presence checks each.
- **N6.** No PDP screenshotting, no headless browser, no review-platform scraping.
- **N7.** No API keys, auth, tenancy, or hosted API. The code is **library-first** (import { scanStore } from "beacon-scan") with the CLI as a thin wrapper, so the shape of an eventual SDK is visible; keys/tenancy/rate plans are designed in docs/part2-architecture.md, not built. Public-data scanning has nothing to authenticate, and a working key system would make this adoptable as-is.

## Assumptions

From docs/PRD.md §7, stated here because a reviewer running this against a different store should know what's being taken on faith:

- **A1.** Public storefront JSON reflects live availability (Shopify serves it from the same catalogue; it can lag by minutes).
- **A2.** Collection order approximates merchandising intent; “bestseller”-style handles approximate demand.
- **A3.** A sold-out variant on a merchandised product is intent lost; the scan cannot see how much.
- **A4.** Detection of back-in-stock capture by HTML markers is approximate — themes vary.
- **A5.** The reviewer runs on macOS/Linux with Node  $\geq 20$ .

## What the scan checks

One headline score (Restock Priority / Demand Leakage) plus a five-stage journey scorecard, all from public data only — /products.json, /collections.json, per-collection product listings, and a throttled sample of sold-out product pages. docs/PRD.md §3 has the full scoring formula and weights; beacon.config.json holds the weights themselves, in one file, so the heuristic isn't buried in code.

| Stage     | What's checked                                                                            |
|-----------|-------------------------------------------------------------------------------------------|
| Discovery | Orphan products (in no collection); sold-out products sitting in top collection positions |

| Stage             | What's checked                                                                                                                                                |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PDP               | Products with fewer than 2 images; short descriptions; colour variants missing an image                                                                       |
| Variant selection | Size-curve holes — whether the <i>middle</i> sizes, not just the edges, are sold out                                                                          |
| Intent capture    | Sampled sold-out PDPs checked for a back-in-stock signup mechanism                                                                                            |
| Trust             | Shipping/refund policy pages present and linked from the homepage — or, if <code>robots.txt</code> disallows checking them, that's stated rather than guessed |

Every finding in the report carries its evidence (endpoint and field) and a recommendation — nothing is asserted without a source.

## Part 2A — From a merchant's question to an answer

I don't let a language model touch a number or write a query. It maps free text to one of a closed set of typed questions, a deterministic engine runs the computation against pre-aggregated tables, and the model only narrates rows it's handed back afterward.

### Four layers, model only at the edges

**1. Question → typed query.** An LLM with structured output maps free text to a closed catalogue of answerable questions — `restock_priority`, `wishlist_to_purchase_leakage`, `price_drop_candidates`, `launch_demand` — each with slots: metric, filters (collection, tag, price band), a time window (30 days default), `top_k`, ranking basis. Off-catalogue questions get “I can't answer that yet; closest I can do is X.” The model never generates free SQL.

**2. Deterministic query engine over per-store intent tables.** Wishlist adds, saves, back-in-stock subscriptions, alert sends, alert-to-purchase conversions, availability, and price history are pre-aggregated daily per product and variant. The typed query compiles to parameterised SQL. `store_id` is bound by the engine itself, not passed in by the prompt — the model never sees or supplies a tenant identifier.

**3. Answer composer.** The response is rows, plus evidence (which signals, which window), a recommended action Swym can execute (send a back-in-stock alert, a wishlist reminder), and a confidence score. The model writes the sentence around the rows; it does not produce the rows.

**4. Clarify or assume.** “Restock first” is ambiguous — by subscriber count, wishlist velocity, or revenue-weighted intent? Default is back-in-stock subscribers plus half the 7-day wishlist velocity, weighted by price, and the answer states that assumption in one line with a toggle to change it.

**Worked example.** For restock priority, the engine ranks variants by  $\text{bis\_subscribers\_30d} + 0.5 * \text{wishlist\_adds\_7d}$ , weights by price, excludes discontinued tags, and returns the top 10 with a line like “14 shoppers waiting, 6 added to wishlist this week, last sold out on March 3.” Every number in that sentence is a column in the row the model was given — nothing is generated.

## Failure modes and safeguards

| Failure mode                                      | Safeguard                                                                                                                                                    |
|---------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hallucinated or drifting numbers                  | Numbers come only from the engine payload; every number in the narration must appear in a row, checked before the answer ships.                              |
| Ambiguous semantics (“best,” “first,” “recently”) | Closed catalogue with a stated default; readings differing by >30% in result set trigger a clarifying question first.                                        |
| Sparse data — most of 50,000 stores are small     | Minimum-sample guard: below ~50 intent events, the answer is “not enough signal yet,” not noise dressed as a ranking.                                        |
| Stale or late pipelines                           | Every answer carries a freshness stamp; time-sensitive questions refuse data older than 24 hours.                                                            |
| Cross-tenant leakage                              | <code>store_id</code> bound inside the query engine, never accepted from the model or prompt; fuzz-tested with adversarial prompts in a standing test suite. |
| Prompt injection via product titles or tags       | Product text is data the model reads, never an instruction it follows; stripped from the prompt, only allow-listed fields reach it.                          |
| Catalogue drift — variants deleted or renamed     | Rows resolve against the current catalogue at answer time; the answer flags it: “3 items no longer exist.”                                                   |
| Cost and latency at 50,000 stores                 | Canonical-question cache per store per day, precomputed top-N tables, a small model for routing, the larger only for narration.                              |
| A merchant acts on a wrong answer                 | Every answer carries evidence, a confidence score, an expandable “why this rank,” and feedback that feeds an eval set.                                       |

Any six of these hold the shape — the constant is that nothing generative touches a number, an identifier, or a query.

## **Delivery surface: API + SDK, dashboard as a client**

The four layers sit behind one intelligence API — `POST /v1/ask`, `GET /v1/insights/restock` — not behind a dashboard. Consumers are Swym’s merchant dashboard, the natural-language chat, Swym’s storefront widgets acting on an answer, headless storefronts, and third-party or agent consumers. A thin JS SDK (@swym/beacon) wraps the API for the common cases. Keys are per store, scoped to what they’re allowed to do (read insights, or also trigger actions), rate-planned, and rotatable. `store_id` is derived from the key on the server — never accepted as a parameter from the caller, since a caller-supplied tenant ID is exactly the cross-tenant leak in the table above. This MVP’s repo mirrors that seam on purpose: `scanStore()` is the library call, the CLI one client of it, a hosted API would be another.

## **Before launching to 50,000 stores**

A golden question-and-answer eval set per store archetype (fashion with sizes, beauty with replenishment cycles, single-SKU brands). A shadow mode that generates answers without showing them and compares them against what an analyst would say. A cohort rollout — 1%, then 10%, then everyone — with a kill switch at each step. Per-answer logging with a hash of the payload it was generated from, so any answer can be replayed later. Cost and abuse budgets per store, so one heavy user doesn’t burn the shared budget. And an “I don’t know” rate tracked as its own health metric — too low is as much a warning sign as too high, since it usually means a refusal guard isn’t firing when it should.

## **Part 2B — Standalone product or a Shopify feature?**

Verdict: as a reporting layer, it’s a feature Shopify will eventually absorb; as an intent-to-action-to-measured-outcome loop, it can stand on its own.

## **Reasoning**

Shopify already has orders, sessions, carts, checkouts, Sidekick, and distribution to every merchant on the platform. Any “what sold well” or “what’s trending in my store” answer is theirs to give, and at some point they will give it — that’s not a hard problem once you already hold the transaction data and the merchant base. What Shopify doesn’t have is pre-purchase intent at the variant level, before the cart: wishlist adds, save-for-later, back-in-stock subscriptions. That’s captured by Swym’s

own widgets on the storefront, across roughly 45,000 stores. Shopify could build the equivalent widgets. They haven't in over ten years. The installed base is the moat here, not the idea behind it.

## Defensible layers, in order of strength

1. **Cross-merchant benchmarks derived from intent.** Category demand curves, size-curve norms, “your back-in-stock conversion is 4% against 11% for comparable stores.” Shopify has little reason to publish cross-merchant intelligence about its own merchants to each other, and merchants want exactly this.
2. **The closed loop.** Beacon says “restock X,” Swym sends the back-in-stock alerts when X comes back, and the system measures the lift. Every answer makes the next one better. A reporting dashboard doesn't do this; an action-and-measurement loop does.
3. **Multi-platform reach.** BigCommerce, Magento, Salesforce Commerce Cloud. Smaller today, but real for the long tail of enterprise merchants who aren't on Shopify at all.

## Distribution as the moat, not just the data

An API and SDK let Beacon sit as the intelligence source inside tools merchants already use — Klaviyo flows, Shopify Flow, headless storefronts, shopping agents. That's harder for a platform to absorb than a dashboard tab, because each integration carries its own switching cost once it's live and other systems depend on it.

## Where it dies

If Beacon is only charts drawn over Swym's own data, Shopify's analytics plus Sidekick turn it into a checkbox feature inside the platform, and there's no reason for a merchant to pay separately for it. The intent data itself is also thin per store for most of the 45,000 — a single small store doesn't generate enough wishlist or back-in-stock signal to be interesting on its own, which is exactly why the benchmark layer, comparing across stores, has to carry the product for small merchants rather than the single-store view.

## Honest counter-argument

Shopify could restrict app data access, or ship a native wishlist and back-in-stock feature that closes the gap Swym's widgets currently fill. That risk is platform-level, not a flaw in this specific product idea, and no amount of engineering here changes it. The mitigation isn't defensive — it's to be the system merchants actually act from, with a lift number they can see and would notice missing. A dashboard is a feature. A loop with a measured outcome attached to it is harder to replace with a checkbox.

# Part 3 — Discovery and prioritisation framework for Beacon

One-sentence answer: find the pain at the moment it happens instead of asking about it afterward, score candidate questions on frequency times severity times Swym's actual data advantage, and kill anything that doesn't lead to a Swym action within two cycles.

## The six steps

1. **Find pain at moments, not in surveys.** Stock-outs, launches, sale weeks, end of season. Talk to 8–10 merchants inside 48 hours of one of those moments and ask what they actually did, in what tool, and what they wish they'd known a week earlier. A survey answered three weeks later describes a memory, not the decision.
2. **Mine what already exists before running new research.** Support tickets, feature requests, which Swym reports or exports actually get pulled and when, which dashboards sit untouched. What merchants do with the tools they already have beats what they say they'd want in a new one.
3. **Score candidate problems on four axes:** frequency, severity (money or time lost), Swym's actual data advantage (does intent data answer this uniquely, or would any analytics tool do it as well), and actionability (does the answer end in something Swym can execute, with a measurable result). Anything Shopify's own analytics answers just as well scores zero on advantage regardless of how it scores elsewhere — that's the filter that keeps the roadmap from turning into a general-purpose BI tool.
4. **Prioritise by learning-per-week, not by feature size.** The smallest version that puts a real answer in front of one design-partner merchant and measures whether they act on it. One question, one store archetype, ten merchants, two weeks — not a quarter-long build for a question nobody's confirmed matters yet.
5. **Ship with kill criteria set before the launch, not after.** Adoption (do merchants come back to it weekly), action rate (did the answer lead to an actual Swym send), and lift (did back-in-stock conversion or wishlist-to-purchase improve). If the action rate is under 20% after two cycles, the question gets retired, not iterated on indefinitely.
6. **Worked example: Beacon's first three questions.** Restock priority (back-in-stock subscribers plus wishlist velocity), price-drop candidates (wishlist, not converting, with a competitive price signal), and launch demand (coming-soon subscribers by variant, before the product even exists in inventory). All three share a property that isn't an accident: each one ends in a Swym send — a back-in-stock alert, a price-drop notification, a launch alert — so the lift is measurable by definition, not by a follow-up survey. A discovery framework that can't point at three questions like this at the end of it hasn't actually converged on anything a merchant would use.